2026 ENTERPRISE EDITION

DATA MODELING & ARCHITECTURE

DATTA SABLE TECHNICAL LIBRARY

Enterprise Data Modeling: Star Schema & Data Vault 2.0

The definitive architecture handbook for designing resilient analytical databases, dimensional star schemas, high-audit Data Vault 2.0 layers, and modern cloud semantic models.

22
Production Chapters & Architecture Blueprints

100%
Practical Code, Schemas & Frameworks

2026
Modern Enterprise Best Practices

INDEPENDENTLY AUTHORED BY

Datta Sable
Enterprise BI & Data Architect

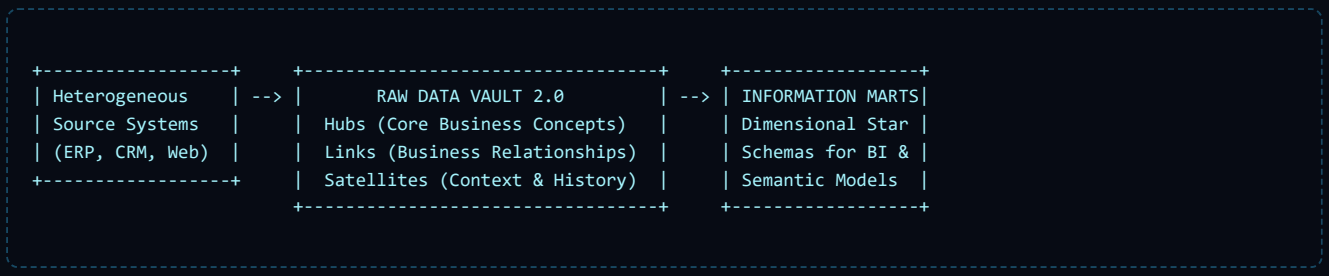
Published: October 2026
dattasable.com/ebooks

ARCHITECTURE BLUEPRINT

Table of Contents & Dimensional Evolution

EXECUTIVE OVERVIEW

Enterprise data modeling has evolved from rigid 3NF relational schemas into agile, dual-tier paradigms: an agile, audit-compliant Raw Data Vault 2.0 back-end coupled with business-facing Kimball Star Schemas for high-performance self-service analytics.



Comprehensive Chapter Guide

Part I: Dimensional Modeling

- Ch 3: Kimball Star Schema Core
- Ch 4: Fact Table Typologies
- Ch 5: Conformed Dimensions
- Ch 6: SCD Types 1 through 6
- Ch 7: Degenerate & Junk Dimensions

Part II: Data Vault 2.0

- Ch 8: Inmon CIF vs Kimball
- Ch 9: Data Vault 2.0 Philosophy
- Ch 10: Hubs, Links & Satellites
- Ch 11: Hash Keys & Hash Diffs
- Ch 12: Business Vault & PIT Tables

Part III: Modern Analytical Patterns: Ch 13 Mart Bridging, Ch 14 Semi-Structured JSON Modeling, Ch 15 Many-to-Many Relationships, Ch 16 Bi-Temporal Modeling, Ch 17 Normalization in MPP, Ch 18 Metric Stores, Ch 19 Healthcare Claims Case Study, Ch 20 Modeling Anti-Patterns, Ch 21 Decision Matrix, Ch 22 Architect Audit Checklist.

CHAPTER 01

Core Principles of Kimball Star Schemas

DIMENSIONAL FOUNDATIONS

Ralph Kimball's dimensional modeling methodology organizes analytical data into single-subject Star Schemas comprising central Fact tables surrounded by denormalized Dimension tables.

A Star Schema prioritizes human understandability and lightning-fast query execution over transactional 3NF normalization. Business users easily navigate single-hop joins between facts and dimensions without traversing multi-table relational mazes.

The Four-Step Dimensional Design Process

1. **Select the Business Process:** Identify the operational activity to model (e.g. Retail Checkout, Claims Ingestion, Loan Approvals).
2. **Declare the Grain:** Explicitly state what a single physical row in the fact table represents (e.g. One individual line-item on a register receipt).
3. **Identify the Dimensions:** Determine the 'who, what, where, when, why' context describing the grain (Customer, Store, Product, Date, Promotion).
4. **Identify the Numeric Facts:** Quantify the event with additive numerical metrics (Quantity Sold, Extended Cost, Net Dollar Amount).

THE GRAIN GOLDEN RULE

Never mix grains in a single fact table. Mixing a transaction line-item with an aggregated daily invoice total leads to double-counting and catastrophic financial reporting errors.

CHAPTER 02

Fact Table Engineering: 3 Fundamental Types

FACT TABLE ARCHITECTURE

Not all business events behave identically. Kimball establishes three distinct fact table typologies to capture continuous business events, periodic balance states, and multi-stage workflows.

Fact Typology	Grain & Timing	Additivity	Primary Use Case
Transaction Fact	Point in time; 1 row per atomic event occurrence	Fully Additive across all dimensions	Retail POS sales, ATM cash withdrawals, web clicks
Periodic Snapshot	Regular intervals; 1 row per entity per period (day/mo)	Semi-Additive (sum across items, not time)	Bank account end-of-day balances, warehouse stock levels
Accumulating Snapshot	Finite lifecycle; 1 row per process instance with date milestones	Fully Additive metrics; updated as stages complete	Order fulfillment (Ordered -> Packed -> Shipped -> Delivered)

Accumulating Snapshot Milestone Schema

fact_order_fulfillment.sql

```sql
CREATE TABLE gold.logistics.fact_order_fulfillment (
    order_id_dd VARCHAR(50), -- Degenerate Dimension
    customer_key INT REFERENCES dim_customer(customer_key),
    order_date_key INT REFERENCES dim_date(date_key),
    packing_date_key INT REFERENCES dim_date(date_key),
    ship_date_key INT REFERENCES dim_date(date_key),
    delivery_date_key INT REFERENCES dim_date(date_key),
    days_to_pack INT,
    days_in_transit INT,
    order_amount_usd DECIMAL(18,2)
);
```

CHAPTER 03

Dimension Table Engineering & Conformed Keys

CONTEXTUAL MASTERY

Dimensions supply the filtering, grouping, and labeling attributes in analytical reports. Designing robust dimensions requires enforcing surrogate keys and enterprise conformed standards.

Natural vs. Surrogate Keys: Never join fact tables to source operational natural keys (e.g. `customer_id = 'CUST-9821'`). Operational keys can change, recycle, or collision-merge across acquisitions. Surrogate keys (synthetic integers: `100421`) provide complete isolation from operational churn.

Conformed Dimensions

A conformed dimension represents an identical entity definition shared across multiple fact tables in an enterprise data warehouse. For example, `dim_date`, `dim_customer`, and `dim_product` must mean the exact same thing whether querying Marketing Facts, Sales Facts, or Logistics Facts.

Role-Playing Dimensions

When a single physical dimension appears multiple times in the same fact table under different business contexts, it 'plays multiple roles'. The Date dimension is the classic example:

- `Order_Date_Key` -> Order Date
- `Shipping_Date_Key` -> Shipping Date
- `Due_Date_Key` -> Payment Due Date

MODELING TIP

In Power BI or SQL, instantiate role-playing dimensions using SQL views or active/inactive relationships with DAX `USERELATIONSHIP()` to preserve single physical storage.

CHAPTER 04

Slowly Changing Dimensions (SCD Types 0 - 6)

TEMPORAL INTEGRITY

Dimension attributes change over time. When a customer relocates from London to New York, how the data warehouse handles this modification dictates historical reporting accuracy.

SCD Type	Mechanism	Historical Preservation	Standard Production Use Case
Type 0	Retain original value; never update	Preserves original state only	Original birth date, original application credit score
Type 1	Overwrite in-place	Zero history (Old value overwritten)	Fixing typos in customer name or spelling errors
Type 2	Insert new row with effective timestamps	Complete, infinite history	Sales territory realignments, customer address relocation
Type 3	Add 'Previous' column to same row	1 previous state only	Tracking immediate prior department or title
Type 6	Hybrid (Type 1 + 2 + 3): 1 + 2 + 3 = 6	Full history + current attribute column	Historical slicing with instant current-state rollups

Standard Type 2 SCD Table Structure

scd_type2_ddl.sql

```sql
CREATE TABLE gold.dimensions.dim_customer (
    customer_sk BIGINT IDENTITY(1,1), -- Surrogate Key
    customer_id VARCHAR(50),           -- Natural Business Key
    customer_name VARCHAR(100),
    region VARCHAR(50),
    effective_start_date TIMESTAMP,
    effective_end_date TIMESTAMP,
    is_current BOOLEAN
);
```

CHAPTER 05

Degenerate, Junk & Outrigger Dimensions

SPECIALTY PATTERNS

Real-world operational systems contain transactional codes, low-cardinality status flags, and secondary hierarchies that do not fit standard flat dimension designs.

Degenerate Dimensions (DD)

An operational identifier (e.g. `Invoice_Number`, `Order_ID`, `Bill_of_Lading`) stored directly in the fact table without a foreign key join to a dimension table. It groups fact rows belonging to a common operational transaction without wasteful dimension overhead.

Junk Dimensions

Combining dozens of disparate, low-cardinality indicator flags (e.g. `Payment_Received`, `Free_Shipping`, `Gift_Wrapped`, `Tax_Exempt`) into a single unified lookup dimension table to prevent fact tables from ballooning with 30 separate foreign key columns.

Outrigger Dimensions

An outrigger is a dimension joined to another dimension rather than directly to a fact table. While strict Kimball rules advise denormalizing attributes into the parent dimension, outriggers are justified when a secondary dimension operates at a distinct grain or is shared across multiple independent dimensions (e.g., a shared County Demographic table joined to both Customer and Store dimensions).

dim_order_flags_junk.sql

```sql
-- Junk Dimension containing cross-product of boolean flags
CREATE TABLE gold.dimensions.dim_order_flags (
    flag_key INT PRIMARY KEY,
    is_gift BOOLEAN,
    is_international BOOLEAN,
    is_discounted BOOLEAN,
    shipping_priority VARCHAR(20)
);
```

CHAPTER 06

Inmon CIF vs. Kimball Dimensional Paradigm

ARCHITECTURAL DEBATE

The two foundational data warehousing philosophies—Bill Inmon's Corporate Information Factory (CIF) and Ralph Kimball's Dimensional Architecture—represent fundamentally different trade-offs between centralized normalization and departmental speed.

Evaluation Dimension	Inmon Corporate Information Factory (CIF)	Kimball Dimensional Architecture
Core Repository	Central 3NF Relational Enterprise Warehouse (EDW)	Decentralized Dimensional Data Marts linked by Bus Matrix
Data Philosophy	Top-Down: Single physical source of corporate truth	Bottom-Up: Subject-oriented analytical business processes
Time-to-Value	Slow; requires enterprise-wide consensus upfront	Rapid; delivers immediate value per business unit
Query Performance	Poor for BI (requires 12-table 3NF SQL joins)	Blazing (Single-hop Star Schema star joins)
Maintenance Burden	High upstream schema rigidity; low redundancy	Moderate redundancy; requires conformed dimension governance

The Modern Consensus: Hybrid Architecture

Modern cloud architectures do not force a binary choice. Enterprise data teams adopt Inmon or Data Vault 2.0 principles for raw historical ingestion and integration, while deploying Kimball Star Schemas in the Gold consumption layer for Power BI, Tableau, and ad-hoc SQL users.

CHAPTER 07

Introduction to Data Vault 2.0: Core Philosophy

ENTERPRISE AGILITY

Invented by Dan Linstedt, Data Vault 2.0 is a detail-oriented, historical tracking, and uniquely linked hybrid modeling methodology designed specifically for enterprise scale, parallel loading, and continuous auditability.

Data Vault isolates business keys, relational associations, and descriptive contextual attributes into three distinct physical primitives: **Hubs**, **Links**, and **Satellites**.

Why Data Vault Succeeds Where 3NF Fails

- **Zero Re-engineering on Schema Evolution:** When source systems add columns or introduce new ERP entities, existing Hubs and Links remain completely untouched. New fields simply attach as new Satellites.
- **100% Auditability & Traceability:** Every row records the exact ingestion timestamp and source system identifier. No data is ever overwritten or deleted.
- **Asynchronous Parallel Ingestion:** By replacing sequence-based auto-increment surrogate keys with deterministic cryptographic hash keys, Hubs, Links, and Satellites load simultaneously across distributed MPP clusters.

CORE PRINCIPLE

A Raw Data Vault should never contain business transformations or cleansing logic. Store source data exactly as received, warts and all, to preserve legal auditability.

CHAPTER 08

Hubs, Links & Satellites: The Core Primitives

PRIMITIVE DEEP DIVE

The structural foundation of Data Vault 2.0 relies on strictly enforcing separation of concerns across identity (Hubs), relationships (Links), and descriptive attributes (Satellites).



Anatomy of the Three Tables

1. **Hub Table:** Contains unique list of business keys. Consists of: Hash Key (PK), Business Key (Natural Key), Load Date Timestamp, and Record Source.
2. **Link Table:** Represents physical associations or transactions between 2 or more Hubs. Consists of: Link Hash Key (PK), Foreign Hash Keys to associated Hubs, Load Date Timestamp, and Record Source.
3. **Satellite Table:** Contains all descriptive, changing attributes. Consists of: Parent Hash Key (FK), Load Date Timestamp, Hash Diff, Descriptive Attributes, and Record Source.

CHAPTER 09

Hash Keys, Hash Diffs & Collision Algorithms

DETERMINISTIC CRYPTOGRAPHY

Data Vault 2.0 utilizes cryptographic hashes (MD5 or SHA-256) to eliminate inter-table loading dependencies, enabling massive parallelization across distributed data warehouses.

Hash Key Construction Rules

Hash keys must be computed identically across all pipelines by applying standard formatting: uppercase trimming, null value substitution, and string concatenation.

hash_key_derivation.sql

```sql
-- Standard DV2.0 Hash Key generation in Snowflake / Databricks
SELECT
    -- Business Hash Key
    SHA2(UPPER(TRIM(COALESCE(customer_id, '-1'))), 256) AS hk_customer_h,

    -- Hash Diff for Satellite change detection
    SHA2(CONCAT_WS('||',
        UPPER(TRIM(COALESCE(customer_name, '^^'))),
        UPPER(TRIM(COALESCE(email_address, '^^'))),
        UPPER(TRIM(COALESCE(postal_code, '^^')))
    ), 256) AS hd_customer_s
FROM staging.crm.stg_customers;
```

Why Hash Diffs Accelerate Ingestion

Instead of comparing 40 individual descriptive columns during CDC ingest to detect changes, the pipeline compares a single 32-byte `hash_diff` column. If the incoming `hash_diff` matches the most recent Satellite record, no write occurs.

COLLISION PROBABILITY

With SHA-256, the mathematical probability of a hash collision across 100 billion records is less than 1 in 10^50—statistically zero for enterprise systems.

CHAPTER 10

Business Vault: PIT & Bridge Tables

QUERY ACCELERATION

Querying Raw Data Vault directly requires joining dozens of Hubs, Links, and Satellites. Business Vault provides Point-in-Time (PIT) and Bridge tables to collapse complex joins into lightning-fast lookups.

Point-in-Time (PIT) Tables

A PIT table tracks snapshot timestamps alongside the foreign key pointers to the exact active Satellite records across multiple Satellites tied to a single Hub. It eliminates slow correlated subqueries when reconstructing point-in-time states.

pit_customer_query.sql

```sql
-- Querying dimensional view backed by a PIT table
SELECT
    pit.snapshot_date,
    h.customer_id,
    s_core.customer_name,
    s_addr.street_address,
    s_addr.city
FROM business_vault.pit_customer pit
INNER JOIN raw_vault.hub_customer h ON pit.hk_customer_h = h.hk_customer_h
INNER JOIN raw_vault.sat_customer_core s_core
    ON pit.hk_customer_h = s_core.hk_customer_h
    AND pit.sat_core_load_date = s_core.load_date
INNER JOIN raw_vault.sat_customer_address s_addr
    ON pit.hk_customer_h = s_addr.hk_customer_h
    AND pit.sat_addr_load_date = s_addr.load_date;
```

Bridge Tables

Bridge tables pre-join complex Link relationships across multiple hops (e.g. Order -> OrderLine -> Product -> Supplier), storing all associated Hub and Link Hash Keys in a single flattened lookup structure.

CHAPTER 11

Bridging Data Vault to Information Marts

CONSUMPTION PATTERNS

Data Vault is designed for ingestion agility and auditing; it is NOT designed for direct end-user reporting. Information Marts translate Data Vault models into dimensional Star Schemas for business tools.

Data Vault Primitive	Maps To in Information Mart	Transformation Complexity
Hub + Attached Satellites	Dimension Table (SCD Type 1 or Type 2)	Low (Flattening attributes via PIT)
Link + Transactional Satellite	Transaction Fact Table	Medium (Joining foreign keys to Dim SKs)
Multi-Hop Links + Bridge Tables	Accumulating Snapshot Fact Table	Medium-High (Tracking milestone timestamps)

Virtual vs. Physical Marts

In modern columnar engines (Snowflake, Databricks, BigQuery), Information Marts can frequently be deployed as **100% Virtual SQL Views** over Business Vault PIT tables. This eliminates data replication and ensures BI dashboards immediately reflect newly committed data without nightly batch delays.

MATERIALIZATION CRITERIA

Materialize Information Mart views into physical Delta or Snowflake tables only when query profiles indicate complex multi-table joins exceed a 3-second interactive dashboard SLA.

CHAPTER 12

Semi-Structured & JSON Modeling Patterns

MODERN POLYMORPHISM

Modern applications transmit data in flexible, nested JSON payloads. Modeling semi-structured data in cloud warehouses requires balancing schema-on-read flexibility with columnar query speed.

The Hybrid Variant Pattern

Instead of creating 50 separate tables for volatile JSON attributes or normalizing nested arrays into deep 3NF joins, cloud warehouses (Snowflake `VARIANT`, BigQuery `JSON`, Databricks `STRUCT/MAP`) allow storing raw payloads alongside typed relational columns.

semi_structured_extraction.sql

```sql
-- Extracting nested JSON attributes in modern analytical queries
CREATE OR REPLACE VIEW gold.analytics.vw_web_events AS
SELECT
    event_id,
    event_timestamp,
    user_id,
    -- Direct dot-notation extraction from VARIANT column
    event_payload:device.os::VARCHAR AS operating_system,
    event_payload:device.browser::VARCHAR AS browser_name,
    event_payload:cart.items[0].product_id::INT AS primary_item_id,
    event_payload AS raw_unparsed_json
FROM silver.events.raw_telemetry;
```

When to Explode vs. When to Retain JSON

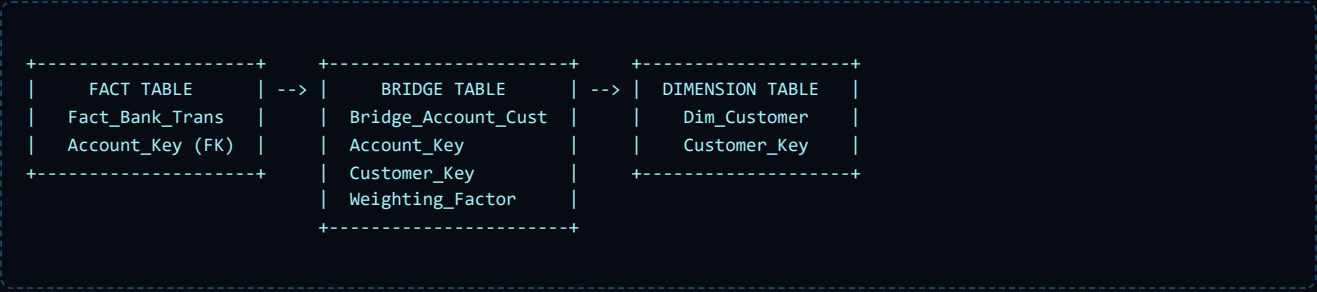
- **Extract to Top-Level Columns:** Any attribute queried regularly in `WHERE` clauses, join predicates, or `GROUP BY` aggregations to leverage physical micro-partition pruning.
- **Retain as JSON/Variant:** Rarely accessed operational debug flags, rapidly evolving external payloads, and sparse attributes present in < 5% of records.

CHAPTER 13

Many-to-Many Relationships & Multi-Valued Dimensions

ADVANCED RELATIONSHIPS

Real-world business entities frequently exhibit many-to-many relationships (e.g. Multiple bank accounts owned by multiple joint customers, or patient visits diagnosed with multiple medical conditions).



Weighting Factors in Bridge Tables

When multiple dimension members share a fact row, simply joining through a bridge table causes row explosion and duplicates financial amounts. Incorporating a **Weighting Factor** (e.g. 0.5 for a 50/50 joint account) allows reports to allocate proportional revenue accurately.

bridge_allocation_query.sql

```sql
SELECT
    c.customer_name,
    SUM(f.transaction_amount * b.weighting_factor) AS allocated_balance
FROM fact_account_balance f
INNER JOIN bridge_account_customer b ON f.account_key = b.account_key
INNER JOIN dim_customer c ON b.customer_key = c.customer_key
GROUP BY c.customer_name;
```

CHAPTER 14

Bi-Temporal Modeling: Assertion vs. State

REGULATORY COMPLIANCE

In financial and insurance domains, historical data models must distinguish between when an event actually occurred in the physical world (Valid Time) versus when the data warehouse learned about it (Transaction Time).

The Two Time Dimensions

- **Valid Time (Business Time):** The interval during which a fact or assertion was true in reality (e.g. An insurance policy effective from March 1 to September 1).
- **Transaction Time (System Time):** The interval during which the record was stored and active in the analytical database (e.g. The system learned of a retroactive cancellation on April 15).

Scenario Question	Time Dimension Required	Query Mechanism
"What was our active policy count on March 15th as we believed it then?"	System Transaction Time	`WHERE recorded_date <= '2026-03-15'` (Reproduces historic audit)
"What was our active policy count on March 15th based on what we know today?"	Valid Business Time	`WHERE effective_start <= '2026-03-15' AND effective_end > '2026-03-15'`

LEGAL DEFENSE

Bi-temporal modeling is the only architectural defense against regulatory audits requiring organizations to recreate financial statements as published on an exact prior historical date.

CHAPTER 15

Normalization vs. Denormalization in Cloud MPP

ENGINE CO-DESIGN

Classical 3NF normalization rules were formulated in the 1970s when disk storage was astronomically expensive and relational databases processed rows sequentially. Modern columnar cloud engines operate on inverted principles.

Why Columnar Engines Love Wide Tables (OBT)

Modern cloud data warehouses (Snowflake, BigQuery, Databricks, Redshift) store data by column, not by row. A wide One Big Table (OBT) with 200 columns incurs zero I/O read penalty when a query requests only 4 columns; the engine reads only those 4 columnar files from storage.

Architecture Pattern	Join Overhead	Storage Footprint	BI Performance
3NF Relational Model	Extreme (10+ multi-table joins)	Minimal (Zero redundancy)	Slowest; prone to join memory spills
Kimball Star Schema	Low (Single-hop foreign key joins)	Low-Medium (Denormalized dims)	Fast; ideal for semantic models
One Big Table (OBT)	Zero (Fully pre-joined flat table)	High (Substantial redundancy)	Blazing for ad-hoc aggregation queries

RECOMMENDATION

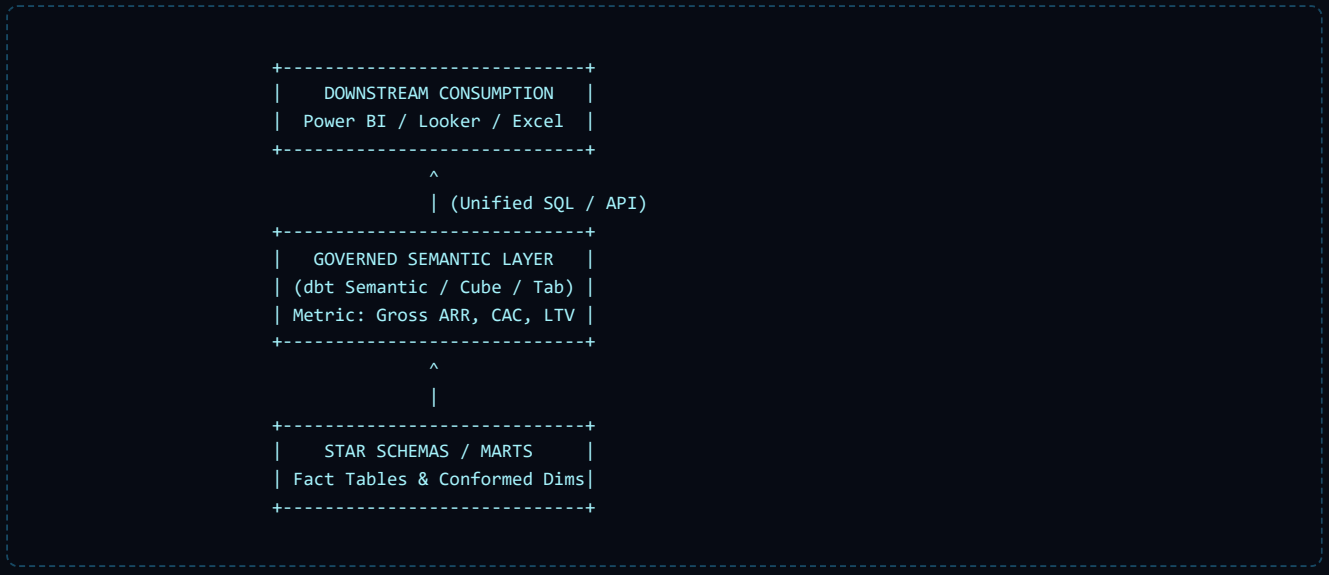
Use Star Schemas as the governed semantic core. Deploy OBT structures for high-concurrency external customer-facing APIs or specialized analytics where join latencies must be eliminated.

CHAPTER 16

Metric Stores & Semantic Layer Modeling

GOVERNED METRICS

When three analysts query the data warehouse for 'Active Customers', they frequently produce three conflicting numbers due to divergent SQL filters. Metric Stores decouple metric definitions from downstream visualization tools.



Core Principles of the Modern Semantic Layer

- **Define Metrics in Code:** Business calculations (e.g. `Net Retention Rate`) are version-controlled in Git repositories rather than hardcoded in isolated Power BI PBIX files.
- **Dynamic Join Traversal:** The semantic engine translates dimension requests into optimal SQL joins at query runtime based on declared data graph relationships.
- **Universal Consumption:** Python data scientists, Excel financial analysts, and BI developers consume the identical metric logic via standard SQL/REST interfaces.

CHAPTER 17

Production Case Study: Healthcare Claims Vault

ENTERPRISE CASE STUDY

A national healthcare provider with 18 million members migrated from a brittle 3NF relational warehouse to an agile Data Vault 2.0 back-end feeding Kimball Star Schemas for clinical and financial analytics.

Architecture Overview & Scale

- **Source Complexity:** 12 independent hospital EHR systems, 4 billing platforms, and pharmacy claims feeds.
- **Raw Vault Core:** `Hub_Patient`, `Hub_Provider`, `Hub_Encounter`, `Link_Encounter_Claim`, and 16 Satellites capturing clinical diagnoses and lab measurements.
- **Information Marts:** Star schemas tracking Readmission Risk Facts and Claims Adjudication Facts.

claims_star_schema.sql

```sql
-- Gold Mart Fact Table generated from Raw Vault
CREATE TABLE gold.claims.fact_adjudication AS
SELECT
    l.hk_encounter_claim_l,
    p.patient_sk,
    pr.provider_sk,
    d.date_key AS service_date_key,
    s_claim.billed_amount_usd,
    s_claim.allowed_amount_usd,
    s_claim.paid_amount_usd,
    s_claim.denial_reason_code
FROM raw_vault.link_encounter_claim l
INNER JOIN business_vault.pit_claim pit ON l.hk_encounter_claim_l = pit.hk_claim_l
INNER JOIN raw_vault.sat_claim_financials s_claim
    ON pit.hk_claim_l = s_claim.hk_claim_l
    AND pit.sat_claim_load_date = s_claim.load_date
INNER JOIN gold.dimensions.dim_patient p ON l.hk_patient_h = p.hk_patient_h
INNER JOIN gold.dimensions.dim_provider pr ON l.hk_provider_h = pr.hk_provider_h;
```

Business Impact: Data onboarding cycles for newly acquired hospital networks dropped from 9 months to 3 weeks. Full HIPAA audit history was preserved with zero data loss.

CHAPTER 18

Data Modeling Anti-Patterns & Common Traps

PITFALL PREVENTION

Flawed dimensional modeling choices made during early architecture phases create compounding technical debt that degrades query performance and distorts business metrics.

Anti-Pattern	Why It Occurs	Architectural Remediation
Snowflaking	Developers normalize dimensions (e.g. Product -> Category -> SubCategory) out of 3NF habits	Denormalize hierarchies into a single flat dimension table. Columnar compression renders repeated strings virtually free.
Fact-to-Fact Joins	Attempting to correlate two business processes directly in a single query	Never join facts directly. Use Conformed Dimensions to drill-across separate star schemas.
Centipede Fact	Fact tables with 40 foreign key joins to micro-dimensions	Consolidate low-cardinality flags into Junk Dimensions. Restrict foreign keys to primary business entities.
Natural Key Leakage	Using operational strings as primary keys across fact tables	Always enforce surrogate keys or deterministic hash keys to protect against source key mutation.

The Danger of Over-Partitioning Facts

Partitioning a fact table by both Country, Year, Month, and Department splits data into tiny, fragmented files. Cloud MPP engines spend more time negotiating file metadata than scanning rows. Limit physical partitioning to 1 or 2 high-cardinality date keys.

CHAPTER 19

Data Modeling Architecture Decision Matrix

METHODOLOGY SELECTION

Selecting the appropriate data modeling methodology depends strictly on organizational complexity, regulatory audit constraints, and data warehouse usage patterns.

Business Constraint	Kimball Star Schema	Data Vault 2.0	One Big Table (OBT)
Frequent Source Schema Changes	Requires Mart Schema Alters	**Superior** (Zero-touch Hubs/Links)	Poor (Full table rebuild)
Regulatory Legal Auditability	Medium (Requires complex SCDs)	**Superior** (100% Raw Lineage)	Poor (Loss of granularity)
Direct Power BI / BI Tool Query	**Superior** (Single-hop joins)	Poor (Excessive joins required)	Excellent (Zero joins)
Multi-Source Master Key Integration	Difficult to reconcile	**Superior** (Hub Business Keys)	Difficult to manage
Implementation Speed	Rapid (Weeks)	Medium (Requires DV automation)	Fastest (Hours)

Architectural Decision Flowchart

- **Do you require strict legal auditing & have 5+ heterogeneous source systems?**
  → Deploy **Data Vault 2.0** in the Core Storage Layer.
- **Are you serving self-service semantic models and executive reporting?**
  → Expose **Kimball Star Schemas** in the Consumption Layer.
- **Are you powering sub-second external customer analytics?**
  → Materialize specialized **OBT Flattened Tables**.

CHAPTER 20

Enterprise Data Architect Master Audit Checklist

PRE-FLIGHT ASSESSMENT

Verify every architectural standard across this master checklist before declaring any enterprise analytical data model production-ready.

- [ ] **Grain Formalization:** The grain of every fact table is explicitly documented in data dictionaries; no mixed-grain facts exist.
- [ ] **Conformed Standards:** Enterprise conformed dimensions (Date, Customer, Product, Org) share identical surrogate keys across all departmental marts.
- [ ] **Surrogate Key Integrity:** All star schema joins execute over synthetic integer surrogate keys; zero operational natural keys exist in join predicates.
- [ ] **SCD Strategy:** Temporal attributes correctly assigned Type 1 (overwrite), Type 2 (effective dating), or Type 6 (hybrid) policies with verified load scripts.
- [ ] **Data Vault Hash Governance:** Cryptographic hash keys formatted with deterministic uppercase trimming and null character handling; zero collision exposure.
- [ ] **Semantic Layer Decoupling:** Business KPIs (ARR, LTV, Churn) defined once in a governed semantic layer rather than hardcoded in front-end dashboards.

AUTHOR CLOSING STATEMENT

Exceptional data architecture survives decades of operational turnover. By anchoring your raw data in the resilient auditability of Data Vault 2.0 and serving business units through intuitive Kimball Star Schemas, your enterprise achieves the perfect equilibrium of agility and analytical precision.

Datta Sable · Enterprise Analytics & Data Architect · *dattasable.com*